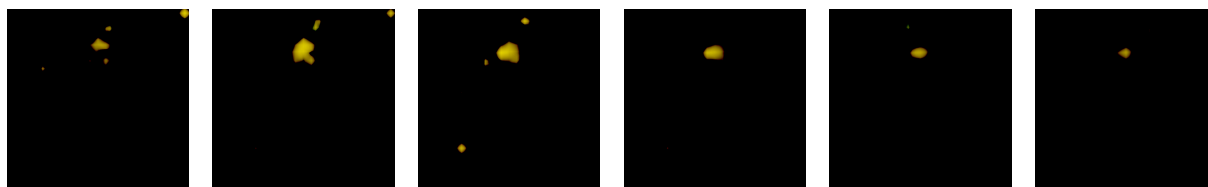


Issues with very small hashgrid network and large hashgrid networks

Validation Testing data context

Small Hashgrid network Levels ($n_levels < 8$)

From the validation testing section, it seemed like $n_levels=2$ or $n_levels=4$ were the best as they had the highest peaks (~29 dB in PSNR) and they didn't have any drastic decreases in them. However, when actually visualizing slices for radiance along the z -axis, there were some striking issues:



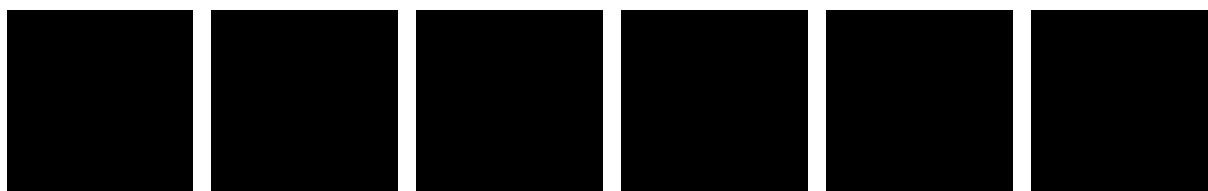
You can see here that there's some structure, but it is very blocky and not high resolution at all, not like what one would expect a flame to look like.

The likely issue here is that the hashgrid is underfitting to my data. Since Instant-NGP represents space as a hierarchy of grids, the low n_levels means that the grid doesn't have a fine enough representation/doesn't have enough levels and is too coarse to fully represent sharper changes within the volume. More of why the validation error was biased or preferred the smaller networks is explained in [validation_error_explanation.pdf](#)

What this means is that there were not enough bins within the grid to learn the higher definition representation of the flame itself, leading to blurry and inconsistent blocks around the image (some due to the fact that multiple points likely mapped to the same coordinates and caused a hash collision)

Larger Hashgrid network levels ($n_levels \geq 16$)

Following the above, since lower n_levels couldn't represent the scene's radiance consistently, what about the higher ones? Well this lead to a different issue, one that isn't just a increasing validation test error (from the graph above)



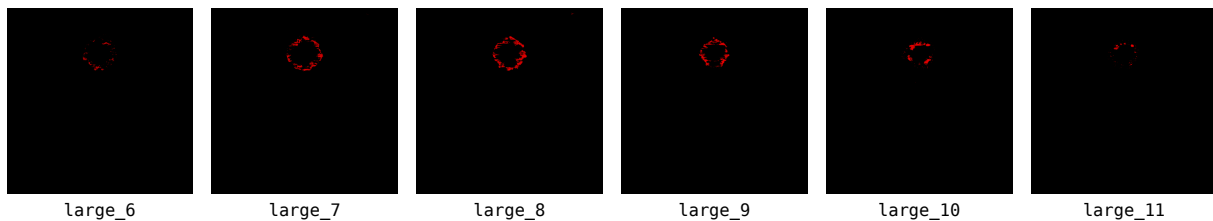
large_no_radiance_5 large_no_radiance_6 large_no_radiance_7 large_no_radiance_8 large_no_radiance_9 large_no_radiance_10

From the images above, you can see they are almost all black/empty, meaning that it predicted little to no radiance coming from the slices.

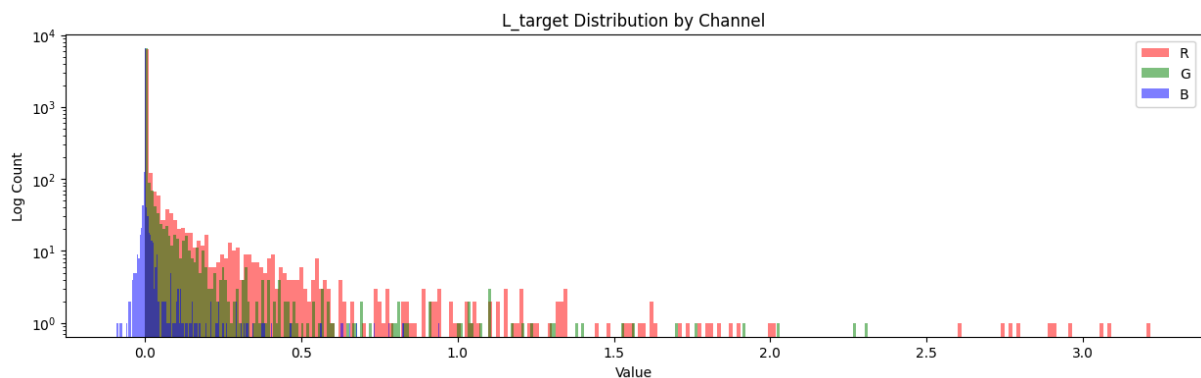
The likely explanation is that with a much larger grid and higher growth factor of 1.5, the finest grid resolution became massive, with loads of different voxels. Therefore, because the volume itself is relatively sparse, the fire itself took up a lot less space comparatively to the amount of empty space in the scene. So, because the grid was so fine there were many hash entries in the hashgrid that simply never got touched or updated to take a "fire" ray samples and the ones that did were drowned out by the empty space.

Another explanation is increasing the size of the decoder MLP and the usage of the custom ReLU activation for radiance. Because the majority of the samples had no radiance and so may have a negative gradient for the majority of the neurons concerning radiance. Thus, it likely lead to dead ReLU, where it predicted 0 radiance for some of the hash grid entries and thus due to the properties of ReLU where an output of 0 can get stuck at that value, causing the network to converge to learning "all-black"/no radiance.

However, black/no radiance did not always occur. Sometimes it would have the correct shape but would have red radiance instead, like below



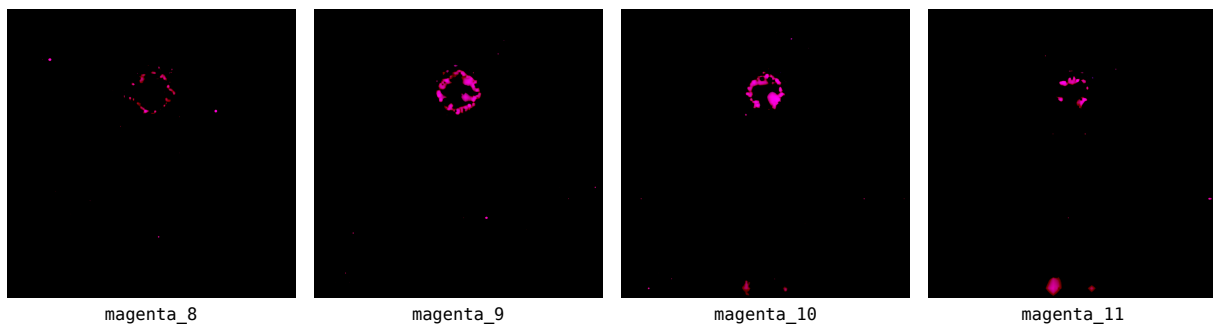
The likely reason for this is similar to the no radiance, but instead since red radiance has a much higher affect on the L2 Loss, the optimizer may have put too much of a importance on the red and thus may have pushed weights responsible for just green/blue to be 0 and similarly experience dying ReLU like above. In fact, the visualized graph of a sample of rays helps to enforce this



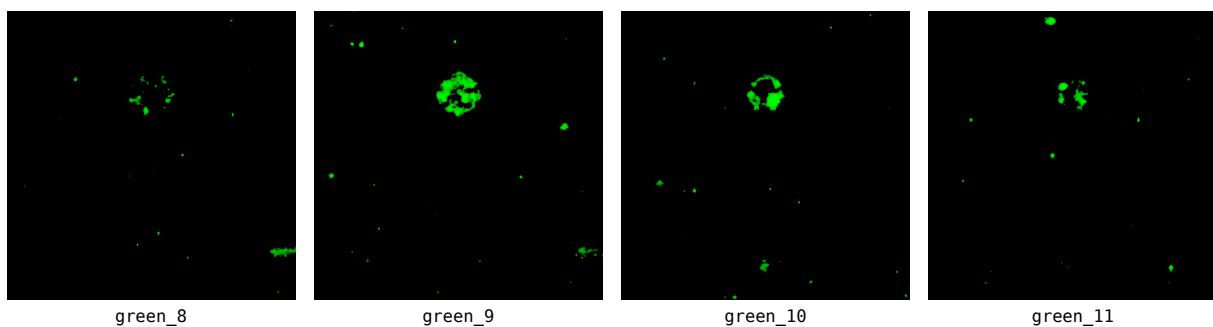
As can be seen above, there are generally more samples with nonzero red radiance, more samples in which the red radiance is a lot bigger than 1 as well. Due to this larger bias towards red, could have caused the network to overfit to learning just red.

Magenta Tangent

This was not the only strange occurrence of a singular radiance value. When testing a configuration with 8 n_levels and log2_hashmap_size of 19 with 64 neurons and 2 hidden layers for the decoder network, this is what I got:



This looks quite strange as the above training visualized graph looks like red should dominate and blue the first one to be reduced to 0. In fact, when combined with smaller decoder networks it can sometimes even get green radiance like below



Here are some potential explanations for the above artifacts

- **Magenta:** At the levels used and for a `log_hashmap` of 19, its possible the spatial grid cells cover a much larger area of the scene. Since the explosion is locally very close and small (only in the center) and green radiance is only really shown in the center of the fire where it is brightest/hottest, it gets averaged across the few grid cells present. Since the histogram has a larger proportion of values of blue close to 0 it does not get as diluted/averaged. Thus, when training its possible that green gets averaged to 0 causing dead ReLU whereas red and blue survive
- **Green:** At even smaller hashgrids and decoder networks, the most fine grid cells at the highest hashmap levels may be too coarse, causing potentially contradictory values to be sampled since too many points map to the same grid cells. Since red has the highest frequency of non-zero radiance and also the highest radiance given, its possible that the L2 loss becomes so volatile that it overshoots and causes it to output 0. Since blue is so low, its possible that the network decided to cut that to by default always be 0 as well and then since green is more tightly bounded, it isn't affected too much and thus leaves an underfitted green blob due to both red and blue being subject to dead ReLU.

Note, the above is just speculative on what caused the strange artifacting of the radiance and may not be the true cause. A more thorough investigation is likely required to fully understand what caused this issue. It's likely it would be better to just split the transmittance and radiance prediction into 2 separate networks to better understand what caused this as it may have been impacted by the transmittance weight updates as well, but this is work for the future. At regular sizes, this isn't a massive problem.

A healthy medium: The goldilocks range (`n_levels=8/12`)

From the previous experiments, its clear that the network can't be too small/dumb but can't be too complex to just learn the empty space, we need to strike a healthy balance between them. I found that this was around `n_levels=8` or `n_levels=12`. However, this is to ensure that it can learn both radiance and throughput. If we want to consider only getting high fidelity results for just throughput or radiance, a larger network is better in this regard. This is why in the final evaluation, only larger networks were considered for evaluation as it only focused on transmittance and not radiance, where a slightly smaller network that would not be affected by radiance issues would be better.